

Mestrax — Restaurante & Bar

Informe de Plan de Pruebas y Aseguramiento de Calidad (QA)

Asignatura: Taller Aplicado de Programación

Institución: Duoc UC

Integrantes: Bastian Olea

Ignacio Lopez

David Albornoz

Introducción

En este documento vamos a detallar todo lo que vendría siendo el plan de pruebas y de seguridad de nuestra aplicación Mestrax, junto a los métodos que usamos, los resultados de las mismas pruebas, y por sobre todo, en la conclusión detallar si la aplicación es viable o no al momento de lanzarla. Todo esto se hizo bajo un criterio de pruebas basado en los requerimientos del restaurante

Dependiendo de cómo sea el resultado de todas estas pruebas, vamos a dar nuestro veredicto como equipo y también las recomendaciones de mejora para el proyecto.

Objetivo de las pruebas

El objetivo de las pruebas es asegurar la calidad de Mestrax. Queremos medir el rendimiento, la estabilidad y la seguridad dentro de la plataforma web para todos los usuarios que la utilicen clientes, garzones y el administrador. La manera en la que logramos hacer estas pruebas es por medio de los métodos de análisis de QA, siendo estos métodos:

- **Caja Negra:** Para probar el flujo que ve el usuario.
- **Caja Blanca:** Para revisar el código por dentro.

Además, como la arquitectura de la aplicación está basada en React y Node.js, hicimos uso de herramientas gratuitas para garantizar que la app no se caiga, siendo la principal para esta prueba usamos jest para el testeo integrado de JavaScript.

Alcance de las Pruebas

Como fue con el proyecto en sí, el alcance de estas pruebas está ligado al tiempo que teníamos para el semestre para poder realizarlas de manera correcta. Además, no es solo por el tiempo, sino que también tiene que ver con la autorización para revisar los datos que maneja la aplicación, así como el análisis a fondo del código de la misma app.

Casos de prueba

Los casos de pruebas los dividimos en diferentes módulos de la aplicación, siendo los más importantes los siguientes:

1. Módulo Login (Accesos de admin y Usuario)
2. Módulo Vista de la Carta / Menú
3. Módulo Carrito de Compras
4. Módulo Formulario de Reserva
5. Módulo Plano Interactivo (Mapa Virtual de las 15 mesas)
6. Módulo Dashboard Admin (Panel de control del administrador)
7. Módulo Backend / API RES

Estrategia o metodología de pruebas

La metodología de las pruebas que le hicimos a la aplicación está basada en 4 tipos de pruebas:

- **Usabilidad:** Con el fin de asegurar la calidad al momento de usar la app, asegurando que la interfaz sea “User Friendly” (amigable para el usuario), intuitiva y atractiva visualmente cuando el cliente interactúa con el menú y el plano de mesas (estos entran en módulos de pruebas no funcionales).
- **Rendimiento:** Para asegurarse de que el nivel de respuesta de la API sea óptimo, optimizando las funciones para evitar tiempos de respuesta largos al cargar los horarios.
- **Estabilidad:** Con el fin de asegurar que, sin importar cuánto tiempo se use la app y la cantidad de usuarios metidos a la vez, esta no tenga problemas de rendimiento ni falle si hay un flujo muy grande de personas reservando en hora punta.
- **Seguridad:** Con el fin exento de evitar inyecciones de código o robos de información sensible dentro de nuestra base de datos.

La manera de realizar estas pruebas es por medio de Caja Negra para evaluar Usabilidad y Rendimiento y Caja Blanca para evaluar la seguridad analizando el código en sectores vitales como el Login o los formularios de reserva.

Defectos detectados

Dentro de este apartado, los defectos que detectamos están más que nada en las pruebas de consola y SonarQube. Defectos que, a pesar de existir, **está comprobado que no afectan en nada al funcionamiento de la aplicación**. Puede que a futuro den algún fallo menor, pero en base a lo que se tiene para el MVP, tenemos bases sólidas al igual que en la seguridad.

Hallazgos de carácter leve a intermedio:

1. **Error en función de filtrado (getReservasByHorario):** El módulo de pruebas detectó una inconsistencia diciendo que la función no era válida dentro de las librerías nativas. El detalle radica en que esta es una función hecha a la medida por nosotros para obtener los datos desde el backend usando una Query. A pesar de que la consola lo resalta como un potencial error, lo testeamos directo en producción y funciona perfecto, no afecta a la aplicación en lo más mínimo.
2. **Advertencia de métodos vacíos (useEffect):** El programa detectó bloques de código o funciones de inicio vacías en las páginas de autenticación y carrito. Esto pasó porque limpiamos código de prueba anterior y quedaron las llaves vacías. No rompe nada, pero la herramienta lo recomienda eliminar para optimizar el código.
3. **Resumen del Test de Compilación:** Al final de la ejecución autónoma, se mostró que de un total de 16 pruebas críticas, **sólo hubo 3 errores**, logrando **16 SUCCESS** en las comprobaciones generales. La aplicación está en perfecto estado para funcionar en producción.

Criterios de aceptación y requisitos Evaluados

Actualmente, el proyecto completo cumple con los estándares de calidad para ser un Producto Mínimo Viable . Evaluamos la usabilidad, estabilidad, rendimiento y seguridad, y todo cumple con los estándares exigidos en las entregas. En cuanto a observaciones, se puede limpiar más el código, y recomendamos que si se quiere escalar el proyecto a un público real o una cadena de restaurantes grande, se planifique pasar el backend actual a una base de datos relacional más robusta a petición del cliente.

Conclusiones

Terminadas todas las pruebas, el proyecto cuenta con una base súper sólida y se puede seguir ampliando para un mayor flujo de usuarios. El enfoque actual funciona excelente tanto para los clientes que entran a la web como para los garzones y administradores del local. El producto final es bastante firme y con total posibilidad de escalar a futuro.

